

ESTRUCTURAS DE DATOS

ÁRBOL BINARIO – TALLER 7

ANGELY NATALIA GAMBOA BERNAL - 55225001

SARA VALENTINA FORERO GUTIERREZ - 55225032

EDITH CATALINA VALBUENA RODRIGUEZ – 55225035

UNIVERSIDAD DE BOYACÁ

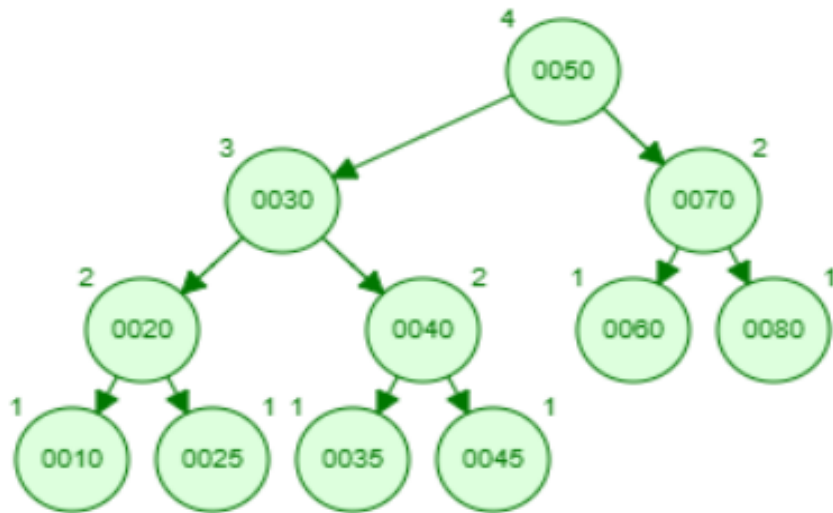
TUNJA

2026

1. VISUALIZACIÓN DE ÁRBOLES GRÁFICAMENTE

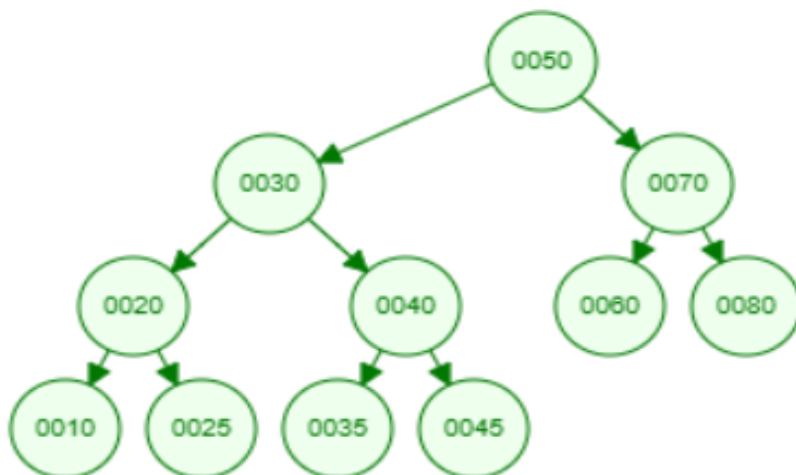
1.1 Propiedades del árbol

1.1.1 Árbol AVL



- Altura y factor de balance: Todos los nodos tiene factor de balance 0 a excepción del nodo 50 que tiene factor de balance 1. Se puede afirmar que el árbol está balanceado.
- Rotaciones necesarias: No hubo ninguna rotación dado que el árbol iba quedando balanceado naturalmente durante las inserciones.
- Niveles: 4 Niveles

1.1.2 Árbol BST



- Niveles: 4 Niveles

¿En qué difieren estos árboles? En este ejemplo no difieren debido al orden de inserción, sin embargo, cabe aclarar que el árbol BST quedó balanceado por casualidad, pero el árbol AVL sí quedó balanceado por regla.

1.2 Recorridos sobre el árbol

- PreOrder: 50, 30, 20, 10, 25, 40, 35, 45, 70, 60, 80
- InOrder: 10, 20, 25, 30, 35, 40, 45, 50, 60, 70, 80
- PostOrder: 10, 25, 20, 35, 45, 40, 30, 60, 80, 70, 50
- LevelOrder: 50, 30, 70, 20, 40, 60, 80, 10, 25, 35, 45

2. RESPUESTAS A LAS PREGUNTAS DE ANÁLISIS

2.1 Preparación del panel y el doble buffer

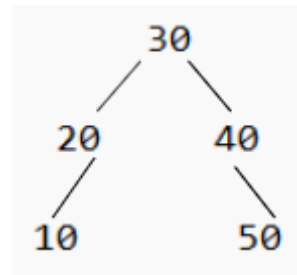
- *¿Por qué es necesario el doble buffer en una aplicación de dibujo recursivo?* Porque al hacer el doble buffer, podríamos decir que se le está “ocultando” al usuario el proceso detrás de lo que él inserta, elimina o actualiza, permitiendo así que el dibujo primero se realice en la memoria temporal y cuando termina si lo muestra completo de una sola vez en pantalla
- *¿Qué ocurre si se omite y se insertan varios nodos rápido?* el panel comienza a parpadear, se verían partes incompletas del árbol, el árbol podría verse borrado por momentos o con líneas que saltan mientras se realiza el proceso, esto se da porque cada inserción hace que el árbol se tenga que redibujar siempre desde cero y mientras más grande se haga el árbol, más evidente será el problema visualmente.

2.2 Asignación de posiciones a cada nodo

- *¿Por qué este método visita primero el subárbol izquierdo antes de asignar la posición del nodo actual?* Porque utiliza un recorrido InOrder (izquierda-raíz-derecha), permitiendo que los nodos menores queden dibujados a la izquierda, el nodo actual en el centro y los nodos mayores a la derecha, así se ven ordenados visualmente.
- *¿Qué representa counter en términos del árbol?* : representa la posición horizontal acumulada en el recorrido, es decir, cuenta cuántos nodos ya se ubicaron y qué tan a la derecha iría el siguiente nodo. Mientras más aumenta el counter, más hacia la derecha aparecerá el nodo que se inserta.

2.3 Posiciones resultantes para un árbol de 5 nodos

Supongamos que tenemos este árbol



Supongamos que tenemos los siguientes valores:

- sideMargin = 50
- spacing = 80
- topMargin = 30
- LEVEL_HEIGHT = 70

NODO 10	NODO 20	NODO 30	NODO 40	NODO 50
Counter = 0 $x = 50 + 0 \cdot 80 = 50$ $y = 30 + 2 \cdot 70 = 170$ Posición =(50,170)	Counter = 1 $x = 50 + 1 \cdot 80 = 130$ $y = 30 + 1 \cdot 70 = 100$ Posición=(130,100)	Counter = 2 $x = 50 + 2 \cdot 80 = 210$ $y = 30 + 0 \cdot 70 = 30$ Posición = (210,30)	Counter = 3 $x = 50 + 3 \cdot 80 = 290$ $y = 30 + 1 \cdot 70 = 100$ Posición =(290,100)	Counter = 4 $x = 50 + 4 \cdot 80 = 370$ $y = 30 + 2 \cdot 70 = 170$ Posición =(370,170)

2.3 Dibujo de aristas y nodos

- ¿Por qué se dibujan primero las aristas y después los nodos? Porque si se dibujaran primero los nodos, visualmente primero aparecen los círculos y luego las líneas de conexión pasarían por encima atravesando los nodos. Caso contrario de lo que realmente pasa y es que al dibujarse primero las líneas, el círculo del nodo se sobrepone haciendo que las conexiones queden por debajo.
- ¿Cómo se calcula la posición del texto para que quede centrado en el círculo? Primero se mide el tamaño del texto con `SizeF sz = g.MeasureString(text, f)` donde text es el valor ingresado y f es el tipo de fuente que se usa, después se usa `g.DrawString(text, f, tb, p.X - sz.Width / 2, p.Y - sz.Height / 2)` para centrar el texto, como se empieza a dibujar desde la esquina superior izquierda, divide entre dos para encontrar la mitad del ancho y la mitad del alto, para luego restarlos y poder centrar el texto tanto vertical como horizontalmente
- Identifica el uso del patrón `using` ¿qué recurso libera en cada caso? El `using` libera recursos gráficos que son nativos de Windows y que dejan de usarse evitando consumo innecesario de memoria o errores de dibujo. En el caso del código en `using (Pen pen = new Pen( ))` libera el objeto pen que sirve para dibujar líneas; en `using`

(*Brush fill = new SolidColorBrush()*) libera el pincel usado para rellenar el nodo; en *using (Font f = new Font())* libera la fuente tipográfica y en *using (Brush tb = new SolidColorBrush())* libera el pincel usado para pintar el texto.

4. BST vs AVL

- ¿Cómo luce el BST? ¿Es útil esa estructura para búsqueda?

El árbol BST quedó completamente desbalanceado hacia la derecha, tomando una forma similar a una lista enlazada y/o degenerado por la derecha. Esto ocurre porque cada nuevo valor insertado es mayor al anterior, por lo que siempre se ubica como hijo derecho. En este caso, el BST deja de ser eficiente para búsquedas porque se deben recorrer muchos nodos secuencialmente.

- ¿Qué altura tiene el BST vs el AVL?

BST: Altura de 10 niveles por el desbalance.

AVL : Altura de 4 niveles.

- ¿Qué complejidad de búsqueda tiene cada árbol?

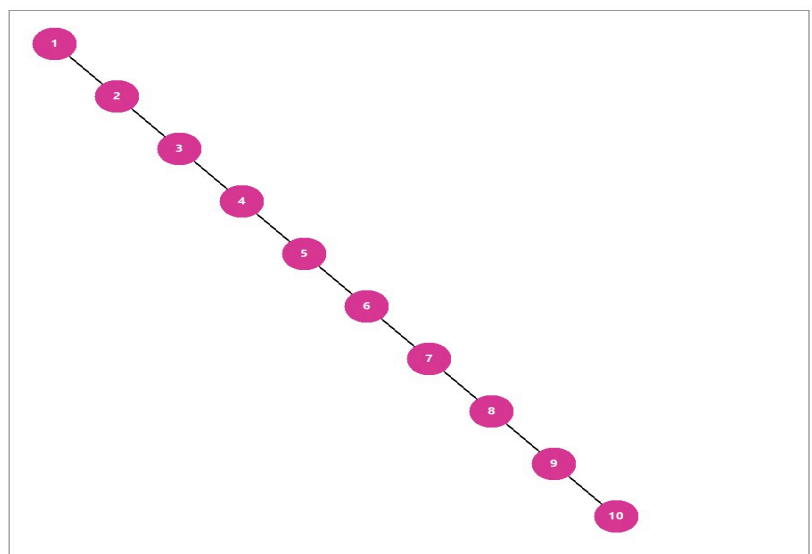
BST desbalanceado: $O(n)$ porque en el peor caso es necesario recorrer casi todos los nodos de manera secuencial.

AVL balanceado: $O(\log n)$ debido a que conserva el equilibrio entre subárboles, reduciendo significativamente la cantidad de comparaciones necesarias.

Árbol BST

Height:
Count:
Min:
Max:

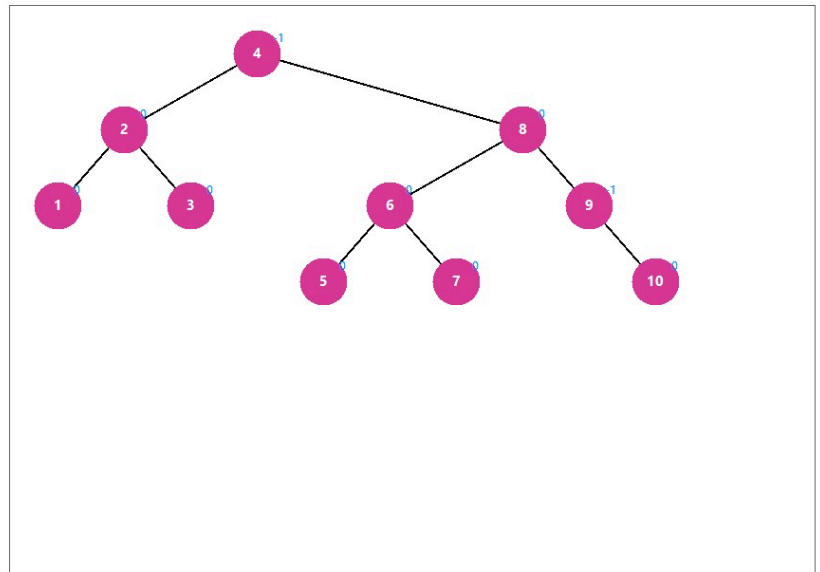
☒ BST ☐ AVL



Árbol AVL

Height:
 Count:
 Min:
 Max:

☐ BST
 ☒ AVL



3. ANÁLISIS COMPARATIVO

El BST puede funcionar de forma eficiente únicamente cuando los datos se insertan en un orden favorable. Sin embargo, puede desbalancear fácilmente y perder rendimiento. Por otro lado, el árbol AVL garantiza un balance automático mediante rotaciones, manteniendo tiempos de búsqueda mucho más eficientes incluso con inserciones desfavorables.

Esto demuestra la importancia de las estructuras auto balanceadas en aplicaciones donde se realizan muchas búsquedas e inserciones.